

A
Project Report
on
**My Education Portal
(Class Management)**

Submitted in partial fulfilment of the
Requirements for the award of the Degree of
BACHELOR OF SCIENCE (INFORMATION TECHNOLOGY)

By
Shyamjanam Prajapati
2543043
Under the esteemed guidance of
Mrs. Sampurna Sarode
Asst.Professor(IT/CS)



DEPARTMENT OF INFORMATION TECHNOLOGY
VPM's RZ SHAH COLLEGE MULUND
(Affiliated to University of Mumbai)
MULUND, 400081
MAHARASHTRA
(2025-2026)

PERFORMA FOR THE APPROVAL PROJECT PROPOSAL

PNR NO:2023016401157481

ROLL NO : 2543043

1. Name of the Students : SHYAMJANAM PRAJAPATI
2. Title of the Project : MY EDUCATION PORTAL
3. Name of the Guide : Mrs. Sampurna Sarode
4. Teaching experience of the Guide: 14 Years

5. Is this your first submission?

Yes

☐

No

☐

Signature of Student

Signature of Guide

Date : ____/____/2026

Date : ____/____/2026

Signature of HOD/Coordinator

Date : ____/____/2026

VPM's R.Z. SHAH COLLEGE OF ARTS, SCIENCE & COMMERCE
NAAC Accredited "A" Grade -First Cycle
(Affiliated to University of Mumbai)
MULUND-MAHARASHTRA-400081



CERTIFICATE

This is to certify that the project entitled," **My Education Portal** ", is benefited work of **Shyamjanam Prajapati** bearing Seat. No: **2543043** submitted in partial fulfilment of the requirements for the award of degree of BACHELOR OF SCIENCE in INFORMATION TECHNOLOGY – Sem VI from University of Mumbai.

Internal Guide

HOD/Coordinator

External Examiner

Date: ____ / ____ /2026

College Seal

Abstract

The **My Education Portal – Class Management System** is a web-based solution designed to streamline academic and administrative processes in educational institutions. Built using **Flask (Python)** for the frontend and **MySQL** for the backend, the system provides a secure and centralized platform for managing student information, attendance, and academic performance. It is divided into two modules: the **Admin Panel**, where staff can add students, record attendance, manage records, and upload marks, and the **User Panel**, which enables students to log in securely to view their details, attendance, and subject-wise performance. By reducing paperwork, minimizing errors, and ensuring transparency, the portal enhances efficiency and accessibility. With its lightweight design and structured data management, the project supports digital transformation in education by delivering a practical, scalable, and user-friendly class management system for administrators and students alike.

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to my project guide, **Mrs. Sampurna Sarode**, for her continuous guidance, encouragement, and valuable insights throughout the development of this project. I am deeply thankful to our respected In-Charge Principal, **Dr. Kavita Sharma**, and the **Head of Department, Prof. Vilas Mahajan**, for providing the necessary infrastructure and resources to complete this project. I also extend my appreciation to the faculty and staff of **VPM's RZ Shah College, Mulund**, for their support. Lastly, I thank my friends and colleagues for their constant encouragement and assistance during this project.

DECLARATION

I hereby declare that the project titled "**My Education Portal**" is my original work and has been carried out under the guidance of **Mrs. Sampurna Sarode**. This project is submitted in partial fulfillment of the requirements for the degree of **Bachelor of Science (Information Technology)** at **VPM's RZ Shah College, Mulund**. I affirm that this work has not been submitted to any other institution or university for the award of any degree or diploma.

I further declare that this project has not been submitted previously to any other university or institution for the award of any degree or diploma. The work presented in this report has been carried out by me during the academic year 2025–2026 as a part of the Semester VI curriculum

SHYAMJANAM PRAJAPTI

2543043

DATE: ____/____/2026

PLAGIARISM REPORT

**0%**
Plagiarism

☐ 0%
Exact Match

☐ 0%
Partial Match

**100%**
Unique

Words	949
Characters	7172
Sentences	50
Paragraphs	12
Read Time	5 minute(s)

Plagiarism Scan Report

Content Checked For Plagiarism

The project My Education Portal is a comprehensive web-based class management system designed to automate and streamline academic and administrative operations within educational institutions. The system is developed using the Flask framework in Python for backend processing, while MySQL is utilized as the relational database management system to ensure structured and efficient data storage. The application follows a modular and scalable architecture, integrating frontend technologies such as HTML, CSS, JavaScript, and Jinja2 templating to provide a responsive and user-friendly interface.

The primary objective of the system is to eliminate the inefficiencies associated with traditional manual methods of managing classroom activities. Conventional systems often involve paperwork and disconnected processes that lead to data redundancy, errors, and delays in accessing critical information. This project addresses these issues by implementing a centralized digital platform that enables real-time data access, improved data integrity, and seamless interaction between administrative staff and students. The system ensures that all academic-related information, including attendance, schedules, and performance records, is stored in a structured database and can be retrieved efficiently whenever required.

The system architecture is divided into two major functional modules: the Admin (Management) module and the Student (User) module. The Admin module is responsible for handling core administrative tasks such as student record management, attendance tracking, performance evaluation, and schedule management. It provides functionalities to add, update, and delete student records, thereby maintaining accurate and up-to-date information. Additionally, administrators can record daily attendance, generate attendance reports, and monitor trends over time. The performance management feature allows entry and modification of academic scores, which are stored in the database and used to generate performance reports for individual students.

The Student module is designed to provide transparency and accessibility to users by allowing students to view their academic information through a secure login system. After authentication, students can access their personal profile, attendance details, class schedules, and performance reports. This module ensures that students remain informed about their academic progress and can track their performance in a structured and organized manner. The separation of modules through role-based access control enhances system security and ensures that sensitive data is only accessible to authorized

Matched Source

Congratulations !
No Plagiarism Found

TABLE OF CONTENT

SR. NO.	INDEX	PAGE NO.
1.	Introduction	11-16
1.1	Background	12
1.2	Objective	13
1.3	Purpose, Scope and Applicability	14-15
	1.3.1 Purpose	14
	1.3.2 Scope	14
	1.3.3 Applicability	15
1.4	Achievements	16
2.	Survey Of Technologies	17-19
2.1	Frontend Technology	18
2.2	Backend Technology	19
2.3	Database Technology	19
2.4	Development Tools	20
3.	Requirements and Analysis	21-31
3.1	Problem Definition	22
3.2	Requirement Specification	23

3.3	Planning and Scheduling	24
3.4	Software and Hardware Requirement	25
3.5	Preliminary Product Description	26
3.6	Conceptual Model	27-31
	3.6.1 Use Case Diagram	27
	3.6.2 Entity Relationship Diagram (ERD)	28
	3.6.3 Class Diagram	29
	3.6.4 Sequence Diagram	30
	3.6.5 Activity Diagram	31
4.	Implementation and Testing	32-47
4.1	Basic Module	33-34
4.2	Data Design	35-37
4.3	Procedural Design	38-40
4.4	User Interface Design	41-42
4.5	Test Case Design	43
5.	Implementation and Testing	45-62
5.1	Implementation Approaches	47-51
5.2	Coding Details and Code Efficiency	52-57
5.3	Testing Approach	58-59

	5.3.1 Unit Testing	58
	5.3.2 Integrated Testing	58
	5.3.3 Beta Testing	59
5.4	Modifications and Improvements	60
5.5	Test Cases	61-62
6.	Results and Discussion	63-69
6.1	Test Reports	64-65
6.2	User Documentation	66-69
7.	Conclusions	70-73
7.1	Conclusion	71
	7.1.1 Significance of the System	71
7.2	Limitations of the System	72
7.3	Future Scope of the Project	73
	References	74
	Glossary	75
	Appendix	76-79

CHAPTER 1

INTRODUCTION

1.1 Background:

In the current era of digital transformation, educational institutions are increasingly relying on technology to streamline administrative and academic processes. Traditional methods of managing class-related activities, such as attendance tracking, student performance evaluation, and data management, are often time-consuming, error-prone, and inefficient. Manual systems tend to result in issues such as lost records, difficulty in accessing real-time information, and limited interaction between teachers and students outside classroom hours. To address these challenges, the development of a web-based class management system has become essential. *My Education Portal* is a comprehensive, user-friendly web application designed using the Flask framework that aims to automate and digitize various aspects of class administration. The system focuses on simplifying and centralizing key functions such as:

- Recording and managing student attendance
- Adding and maintaining student and teacher profiles
- Storing and organizing academic performance and assessment data
- Providing dashboards for quick insights and reporting
- Enabling secure access to academic data for different user roles (e.g., admin, teacher, student)

By implementing such a system, educational institutions can improve operational efficiency, ensure better data integrity, and enhance the overall learning experience for students. Additionally, the system promotes transparency and accountability by maintaining accurate and up-to-date records that can be accessed anytime from any device with internet connectivity. The use of Flask as the core development framework offers several advantages such as rapid development, lightweight design, and ease of integration with databases and front-end technologies. The modular and scalable nature of Flask makes it an ideal choice for building a dynamic educational platform that can evolve with the growing needs of the institution.

In conclusion, *My Education Portal* is not just a tool for administrative convenience, but a step toward fostering a more digital, interactive, and data-driven academic environment.

1.2 Objectives:

The main objective of *My Education Portal* is to create a web-based class management system using **Flask** (Python web framework) and **MySQL** as the database. The system aims to streamline and digitize essential academic and administrative functions of a classroom environment. The core objectives include:

- **Digital Attendance Management:**
 - Enable Staffs to mark and manage student attendance efficiently.
 - Store attendance records securely in a MySQL database for reporting and analysis.
 - Provide attendance summaries for management and students.
- **User Management:**
 - Allow administrators to add, update, or delete users (students, admin).
 - Implement role-based access to control features according to user type.
 - Secure login system for each user role.
- **Academic Data Management:**
 - Enable teachers to upload and maintain student performance data (marks, grades, test results).
 - Store all academic records in a structured database for easy retrieval.
- **Classroom Data Centralization:**
 - Maintain a centralized system for managing all classroom-related information such as subjects, timetables, and student records.
- **Interactive Dashboards:**
 - Provide different dashboards for students, and admins to view relevant data (e.g., attendance reports, academic performance).
 - Use visual elements (charts/tables) for better understanding of classroom statistics.
- **Web-Based Accessibility:**
 - Make the portal accessible via any web browser, allowing students and teachers to access their data anytime, anywhere.
- **Security and Data Integrity:**
 - Ensure secure communication between users and the server using authentication and data validation.
 - Prevent unauthorized access to sensitive student and class information.
- **Scalability for Future Enhancements:**
 - Design the system architecture in a way that allows for future additions such as assignment uploads, or notification features.
 - Features such as online exams, messaging between users, or payment management.

1.3 Purpose, Scope, and Applicability

1.3.1 Purpose

The purpose of *My Education Portal* is to design and develop a web-based class management system that digitizes and automates essential academic and administrative tasks in an educational environment. This system is intended to improve efficiency, accuracy, and accessibility of classroom-related data. Key purposes include:

- **Streamlining Attendance Management:** Automate daily attendance tracking to reduce manual errors and save time.
- **Simplifying User Management:** Provide a secure and easy way to add and manage users including students, teachers, and administrators.
- **Maintaining Academic Records:** Centralize academic data such as grades and assessments for easy access and maintenance.
- **Enhancing Accessibility:** Allow authorized users to access the portal anytime, anywhere through a web interface.
- **Improving Data Security:** Protect sensitive information with role-based access control and secure authentication.

1.3.2 Scope

The scope of *My Education Portal* defines the boundaries and extent of the project. The system will primarily focus on the following areas:

- **Attendance Tracking:** Recording, storing, and reporting student attendance for individual classes.
- **User Account Management:** Creating, updating, and deleting user profiles with role-based permissions.
- **Academic Data Management:** Recording student marks, grades, and other performance indicators.
- **Dashboard and Reporting:** Providing tailored dashboards for different user roles displaying relevant information.
- **Database Integration:** Using MySQL as the backend database for efficient data storage and retrieval.
- **Web-based Access:** Accessible via standard web browsers to facilitate remote access by users.

1.3.3 Applicability

The applicability section describes who and how the system will be used in real-life scenarios:

- **Educational Institutions:** Coaching centers can use the portal to automate their classroom management.
- **Staff:** Can use the system to manage attendance, upload academic data, and monitor student performance efficiently.
- **Students:** Can view their attendance records, academic progress, and other relevant information.
- **Administrators:** Can manage users, oversee class activities, and generate reports for institutional analysis.
- **Remote Access:** The portal's web-based nature allows access from any device with internet connectivity, supporting flexible learning environments such as online and hybrid classes

1.4 Achievements

The *My Education Portal* project aims to achieve the following significant milestones and benefits upon successful implementation:

➤ **Automated Attendance Management**

- Developed a reliable and user-friendly attendance system that allows Staffs to mark and track student attendance digitally.
- Reduced manual errors and saved administrative time by automating attendance record-keeping.

➤ **Efficient User Management**

- Created a secure user registration and login system with role-based access control for students, and administrators.
- Enabled easy addition, modification, and deletion of user profiles to maintain accurate user data.

➤ **Centralized Academic Data Maintenance**

- Designed a database structure in MySQL to store, retrieve, and update student academic records such as marks and grades.
- Provided a systematic approach to managing student performance data accessible to authorized users.

➤ **Interactive Dashboards and Reports**

- Implemented customized dashboards that display attendance summaries, academic progress, and other relevant class information.
- Enhanced decision-making with clear and concise visual reports for administrators.

➤ **Web-Based Accessibility**

- Delivered a responsive web application accessible from any internet-enabled device, supporting remote learning and data access.
- Improved communication and transparency between students and administrators through timely availability of information.

➤ **Secure and Scalable System**

- Ensured data security through authentication, authorization, and validation mechanisms.
- Built the system with scalability in mind to support future feature additions and growth in user base.

CHAPTER 2

Survey of Technologies

2.1 Frontend Technologies

The frontend is responsible for what the user sees and interacts with. A clean, responsive, and user-friendly interface improves usability and user experience.

2.1.1 HTML (HyperText Markup Language)

HTML is the foundation of all web pages. It is used to structure and organize content on the web. In this project:

- HTML is used to design the layout of forms (such as login, registration, and attendance forms).
- It defines elements like text boxes, dropdowns, buttons, tables, and image placements.
- Each web page (e.g., dashboard, student list, marks entry page) is built using HTML structure.

2.1.2 CSS (Cascading Style Sheets)

CSS is used to style HTML elements and control the layout, design, and visual appearance of the portal.

- It handles fonts, colors, margins, padding, backgrounds, and layouts.
- In this portal, CSS is used to make the interface responsive across devices such as desktops, tablets, and smartphones.
- CSS classes and IDs are used to apply consistent styles across multiple pages.

2.1.3 JavaScript

JavaScript brings interactivity to the web pages and allows dynamic changes without reloading the page.

- It performs client-side form validation (e.g., checking empty fields before submitting).
- Enables real-time updates such as live attendance status or dynamic table filtering.
- JavaScript is also used to enhance dashboards with chart libraries (e.g., Chart.js).

2.1.4 Jinja2 (Flask Template Engine)

Jinja2 is a powerful templating engine that works with Flask to render dynamic web pages.

- It allows embedding Python code in HTML using `{% %}` and `{{ }}` syntax.
- Makes it possible to display real-time data (e.g., student marks, attendance % from the database).
- Supports template inheritance to create reusable layouts (e.g., header, footer, navigation).

2.2 Backend Technology

The backend handles all the application logic, data processing, and interactions with the database. It is responsible for processing user requests and delivering responses.

2.2.1 Python (Flask Framework)

Flask is a micro web framework written in Python that is simple, lightweight, and ideal for educational projects.

Why Flask was chosen:

- **Lightweight and Modular:** Flask allows developers to add only the features they need, making the system faster and easier to manage.
- **Routing:** Flask handles URL routing, i.e., mapping a URL (e.g., /login) to a specific Python function (login()).
- **Form Handling:** Flask supports form submission and validation using extensions like WTForms.
- **Session Management:** Flask allows storing login sessions to manage user authentication.
- **Integration with MySQL:** Flask can easily connect to databases through libraries like Flask-MySQL, PyMySQL, or SQLAlchemy.

Use in project:

- Routes and endpoints are defined to handle requests for login, registration, adding attendance, entering marks, and generating reports.
- Python functions are used to validate inputs, fetch and update data in MySQL, and render output to HTML using Jinja2.

2.3 Database Technology

The database is a central part of the system where all the structured data is stored and managed.

2.3.1 MySQL (Structured Database System)

MySQL is an open-source relational database management system (RDBMS). It is widely used for web applications due to its performance, simplicity, and scalability.

Why MySQL was selected:

- **Data Integrity:** Ensures reliable and consistent data through constraints and transactions.
- **Performance:** Can handle large volumes of queries quickly, which is crucial for attendance and performance reports.
- **Compatibility:** Easily integrates with Python through multiple connectors.

Use in project:

- Tables are created for students, teachers, attendance, marks, fees, users, and roles.

2.4 Development Tools

These tools support code writing, debugging, version control, and database management.

2.4.1 Visual Studio Code / PyCharm

These are code editors and integrated development environments (IDEs) that support Python and web technologies.

- **VS Code:** Lightweight editor with powerful extensions for Flask, MySQL, and Git.
- **PyCharm:** Full-featured IDE specifically designed for Python development with features like autocomplete, debugging, and version tracking.

2.4.2 XAMPP

XAMPP is a package that includes Apache (web server), MySQL (database), and PHP.

- Although Flask handles backend logic in Python, XAMPP is used to run and manage the local MySQL database during development.

2.4.3 GitHub

GitHub is a cloud-based version control system.

- It allows saving code versions, tracking changes, and collaborating with others.
- Important for backing up project files and code sharing with mentors or teammates.

CHAPTER 3

Requirement and Analysis

3.1 Problem Definition

Managing class-related activities such as maintaining student records, tracking attendance, scheduling classes, and monitoring academic performance is often performed manually in many educational institutions. This manual approach is time-consuming, error-prone, and lacks transparency for students.

Students frequently face difficulties accessing their academic data, attendance records, and schedules on time, leading to miscommunication and delays. Similarly, the management finds it challenging to keep all student-related data organized, updated, and accessible efficiently.

The absence of a centralized, automated system results in:

- Inefficient data management and retrieval.
- Increased chances of human errors in attendance and performance tracking.
- Lack of real-time access to academic data for students.
- Extra workload on administration due to repetitive manual tasks.

Therefore, there is a need for a **centralized class management system** that allows the management to efficiently handle student data while providing students a transparent platform to access their academic information.

3.2 Requirements Specification

The requirements specification defines the functional and non-functional needs of the proposed **MyEducationPortal** system. These requirements ensure the successful development and deployment of the project.

Category	Sub-Category	Requirements
	Admin (Management) Module	- Add, update, and delete student records.- - Mark and manage student attendance.- - Create, edit, and view class schedules.- - Record and track student performance.- - Secure login authentication for admin users.
Functional	Student (User) Module	- View personal profile details.- - Access attendance records.- - View class schedules.- - Check academic performance.
	Authentication	- Separate login system for students and admin.- - Password-protected accounts for secure access.
	Database Management	- Store all student, schedule, attendance, and performance data in a centralized database (MySQL).
	Performance	- System should provide quick response times for data retrieval and updates.
	Scalability	- Capable of managing increasing numbers of students and records without performance degradation.
Non-Functional	Security	- Protect sensitive student data with secure login credentials and restricted access.
	Usability	- Easy-to-navigate interface for both admin and student users.
	Reliability	- Ensure data consistency and minimize downtime.

3.3 Planning and Scheduling

The development of **MyEducationPortal** will follow a systematic approach to ensure timely and efficient completion. The project is planned using a phased methodology that includes analysis, design, development, testing, and deployment. Each phase has defined tasks and deliverables to ensure smooth project progress.

Project Planning Phases

1. Requirement Analysis (Week 1–2):

- Understand the scope of the project.
- Collect functional and non-functional requirements.
- Identify the roles of admin and student modules.

2. System Design (Week 3–4):

- Design the database schema for student, attendance, schedule, and performance data.
- Create wireframes and UI layout for the web application.
- Define the overall architecture using Flask and MySQL.

3. Development (Week 5–8):

- Implement admin functionalities: add/remove/modify student data, attendance, performance, and schedule management.
- Develop student module: view attendance, performance, and schedules.
- Integrate secure authentication for admin and student.

4. Testing and Debugging (Week 9–10):

- Perform functional testing of all modules.
- Fix bugs and optimize the system for performance.
- Conduct user acceptance testing.

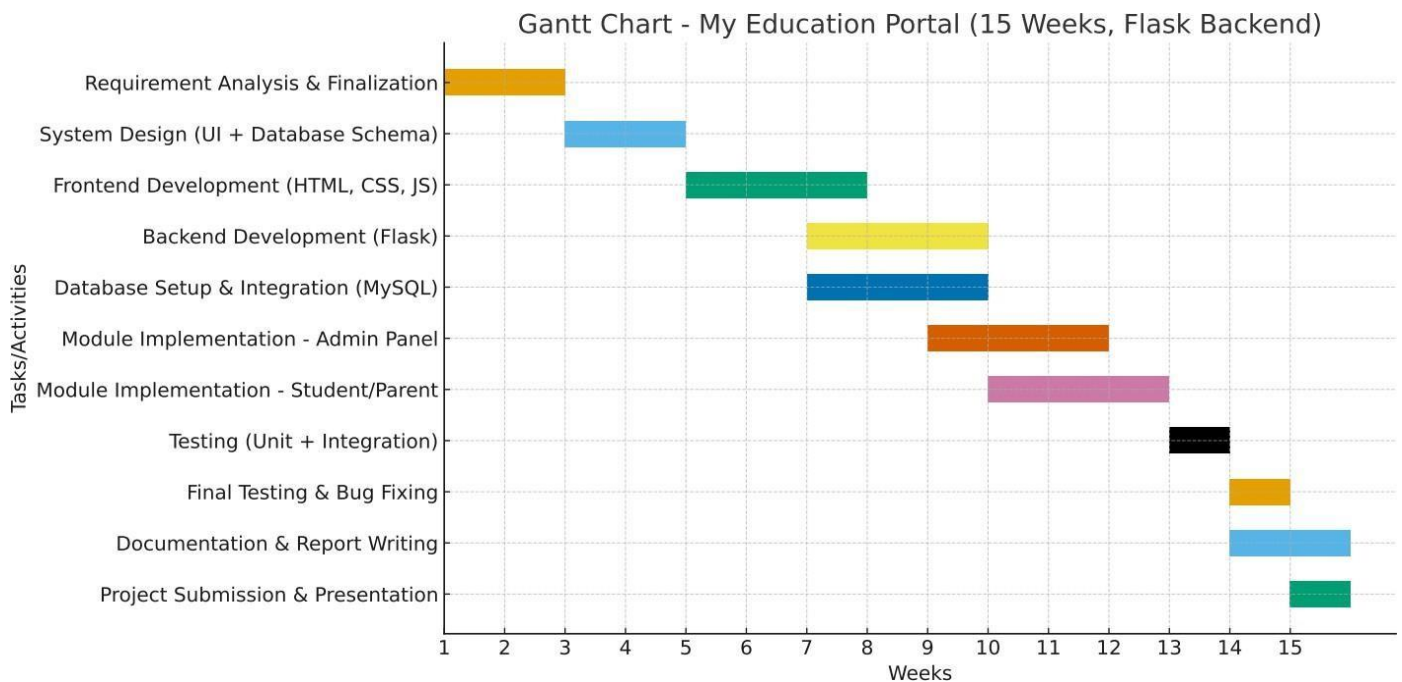
5. Deployment and Documentation (Week 11–12):

- Deploy the system on a local or online server.
- Prepare user manual and project documentation.
- Final presentation and demonstration.

Tools for Planning

- **Project Management:** Gantt chart for timeline tracking.
- **Version Control:** Git/GitHub for code management.
- **Testing Tools:** Manual testing with sample data.

Gantt Chart:



3.4 Software and Hardware Requirements

3.4.1 Hardware Requirements

- Processor: Intel i3 or above
- RAM: Minimum 4 GB
- Storage: 500 GB HDD or above
- Operating System: Windows / Linux

3.4.2 Software Requirements

- Backend: Python with Flask framework
- Database: MySQL
- Frontend: HTML, CSS, JavaScript
- Web Server: Flask Development Server / Apache (optional for deployment)
- Browser: Any modern web browser (Chrome, Firefox, Edge)

3.5 Preliminary Product Description

The **MyEducationPortal** is a web-based class management system designed to streamline academic operations and improve communication between students and educational management. The system is divided into two major modules: **Admin (Management)** and **Student (User)**.

Admin (Management) Module

- **Student Data Management:** Add, modify, and delete student details.
- **Attendance Management:** Record and track attendance for each student.
- **Class Schedule Management:** Create, update, and assign schedules for different classes.
- **Performance Tracking:** Maintain records of student academic performance and progress reports.
- **User Authentication:** Secure login system for management to ensure data privacy and security.

Student (User) Module

- **View Personal Data:** Access personal profile details maintained by the administration.
- **View Attendance:** Check attendance status and history.
- **View Class Schedule:** View assigned schedules for classes and events.
- **View Academic Performance:** Track academic performance updated by the administration.

Technology Stack

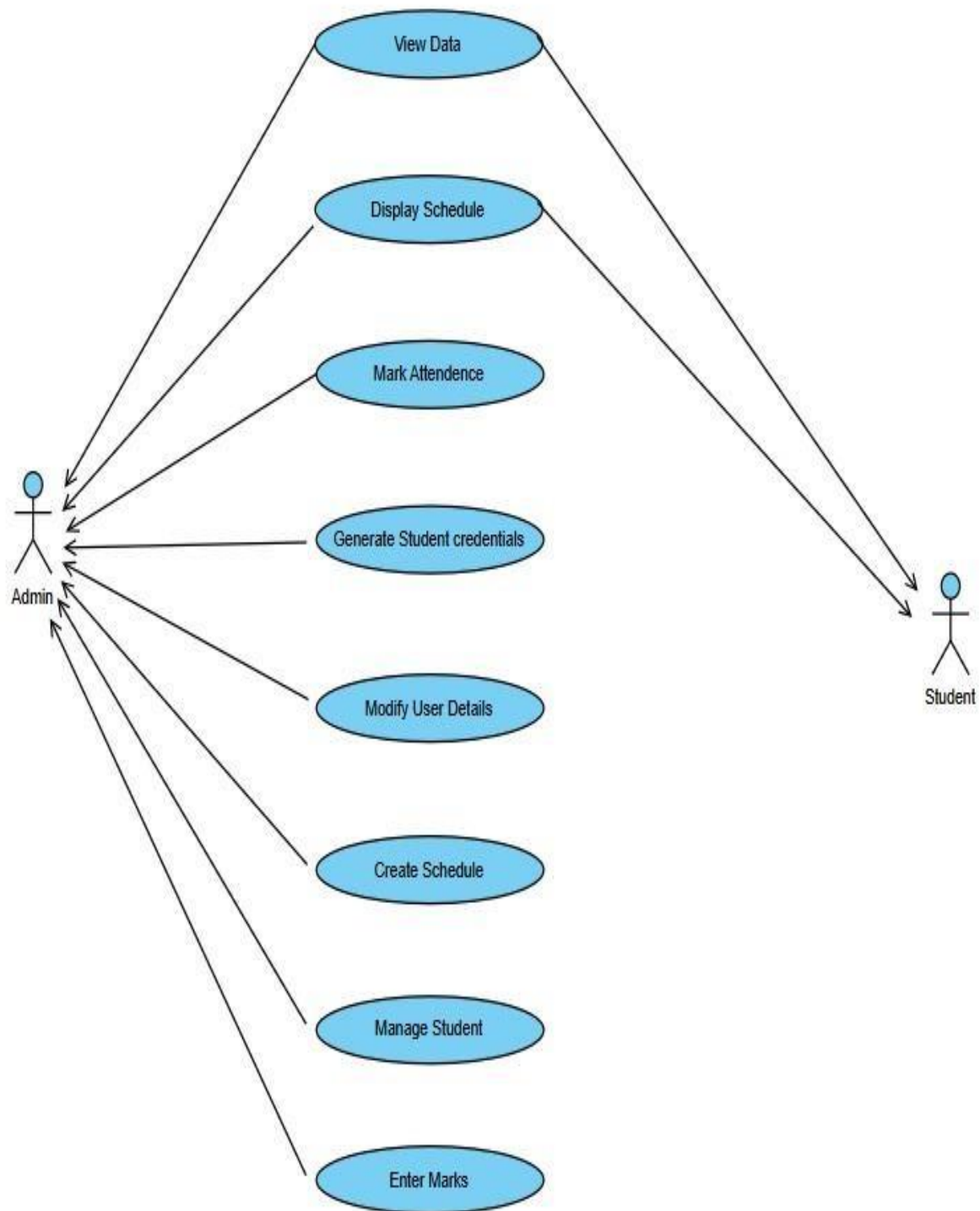
- **Frontend & Backend Framework:** Flask (Python)
- **Database:** MySQL
- **Frontend Technologies:** HTML, CSS, JavaScript (for interactivity)
- **Server:** Flask Development Server (with deployment scope on Apache/Nginx)

Key Characteristics

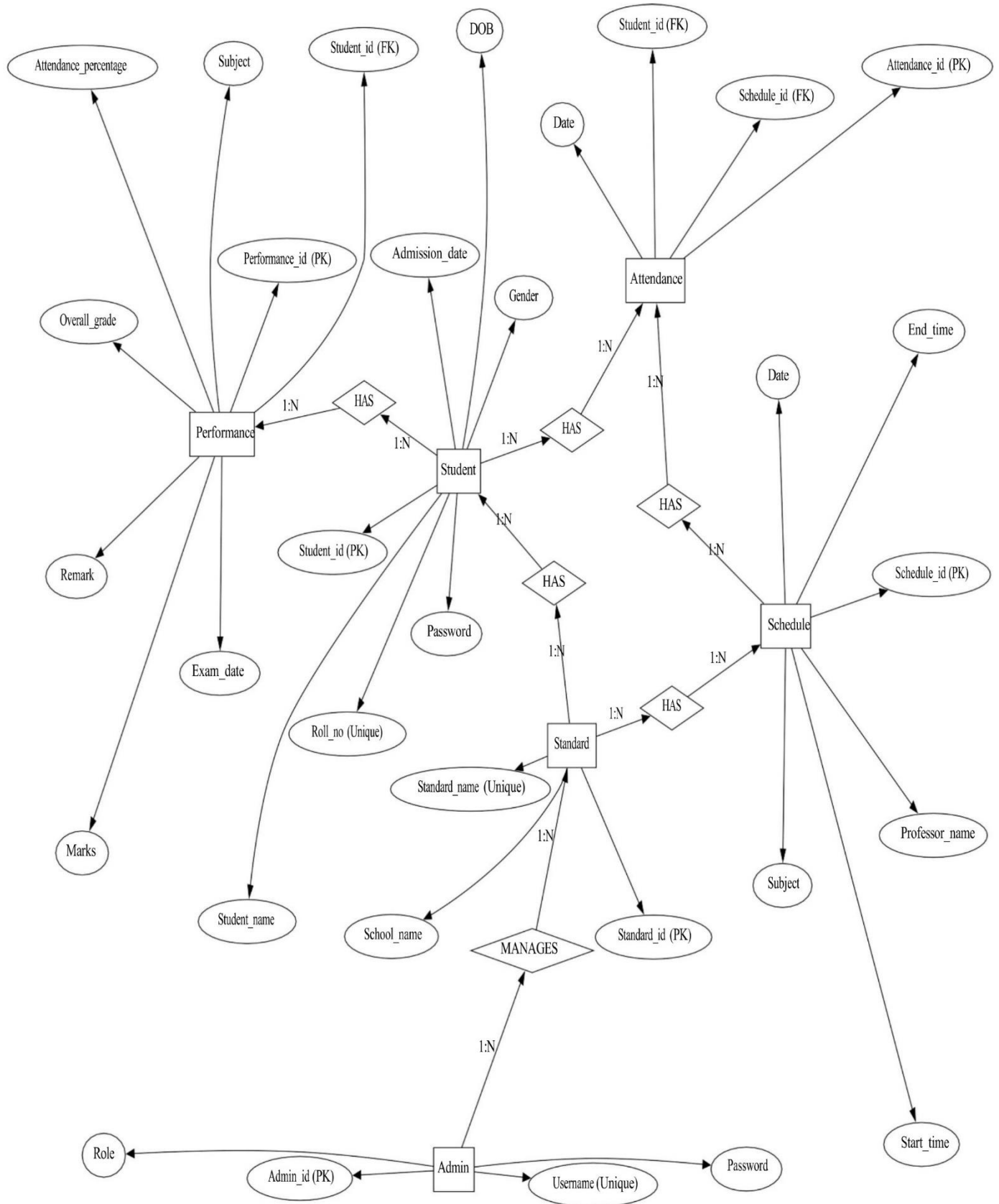
- User-friendly interface with clear navigation for students and management.
- Centralized database ensuring quick data retrieval and updates.
- Secure login system preventing unauthorized access.
- Scalable and adaptable for future integration (notifications, result printing, mobile app).

3.6 Conceptual Models:

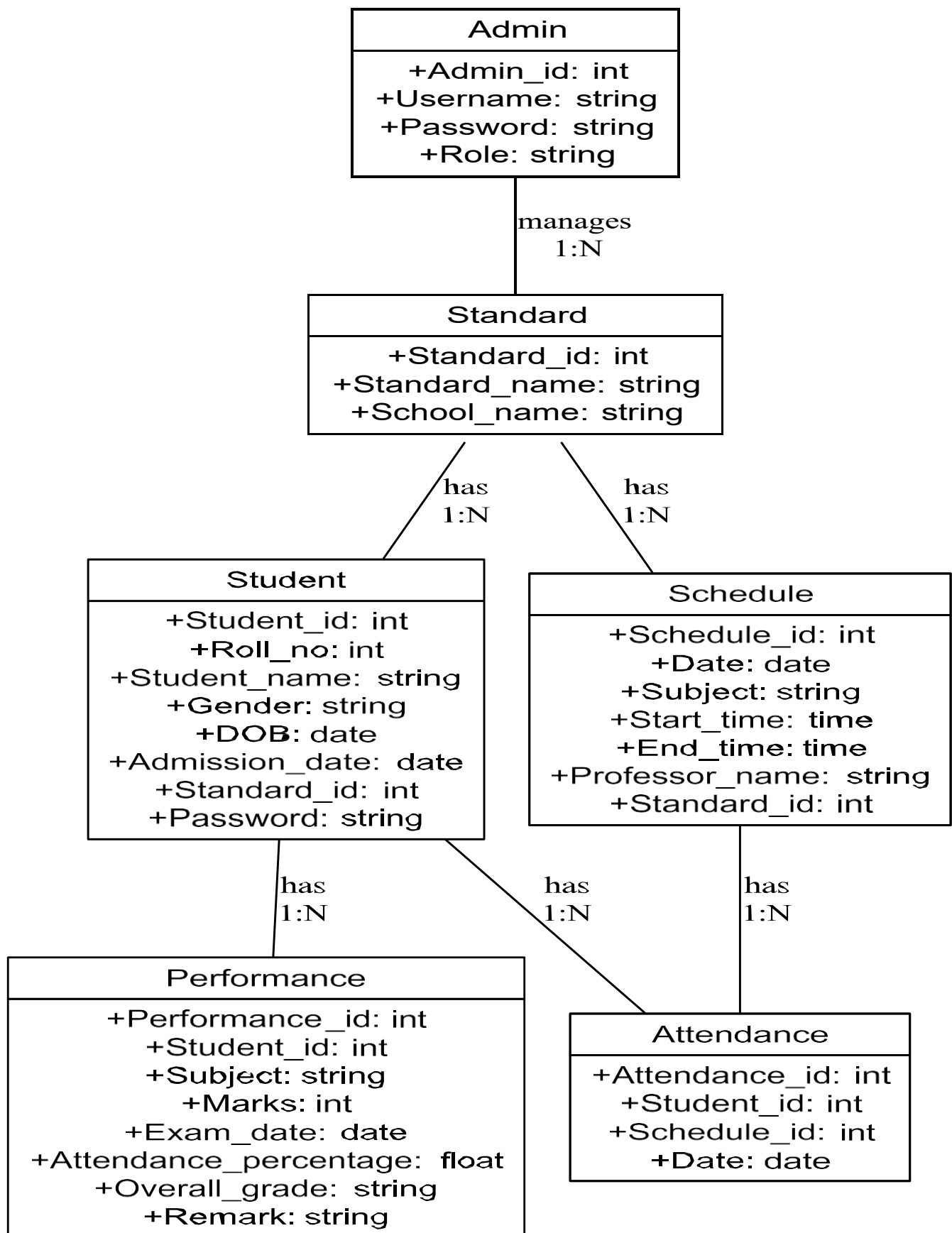
3.6.1 Use Case Diagram



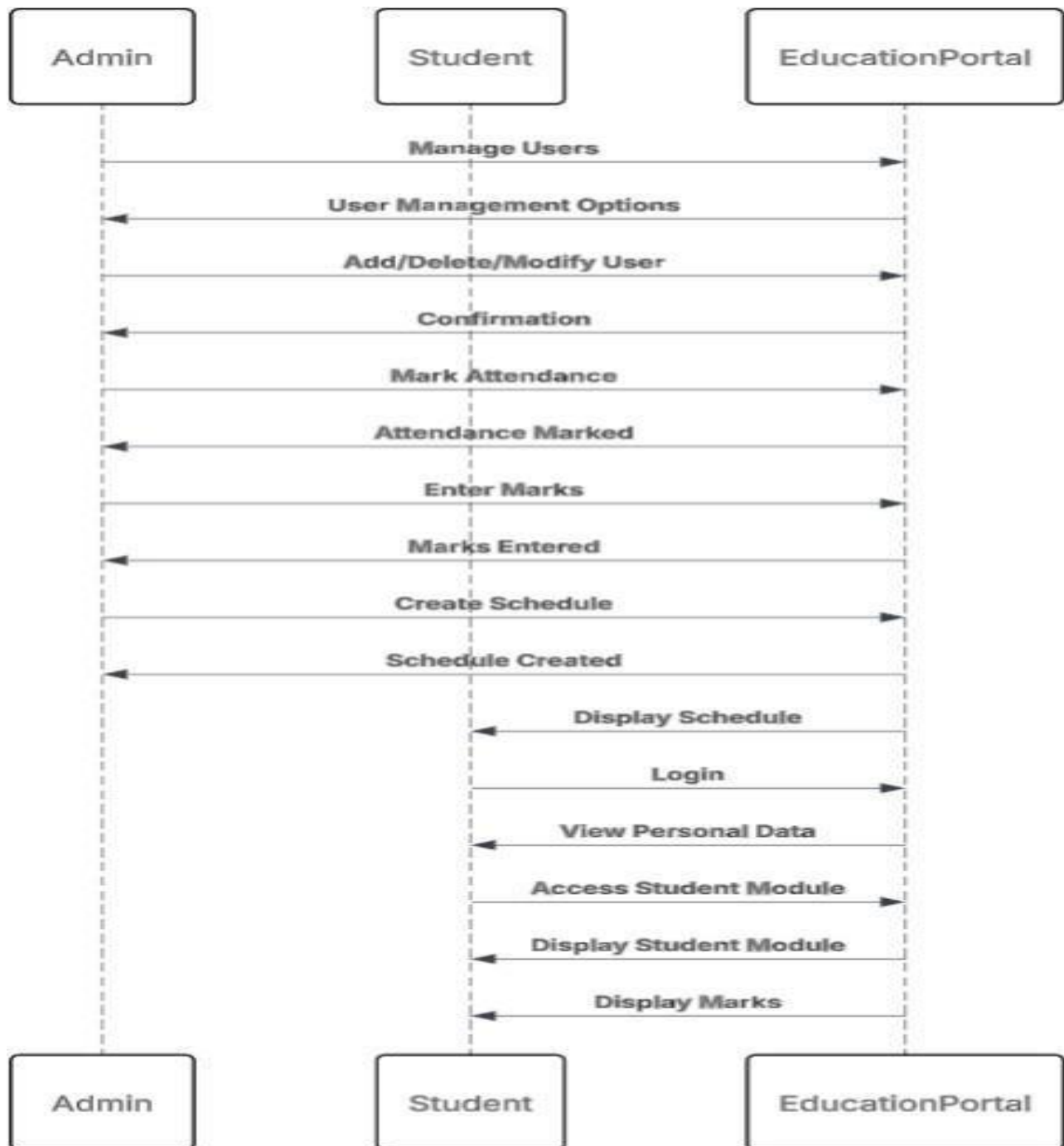
3.6.2 Entity Relationship Diagram (ERD)



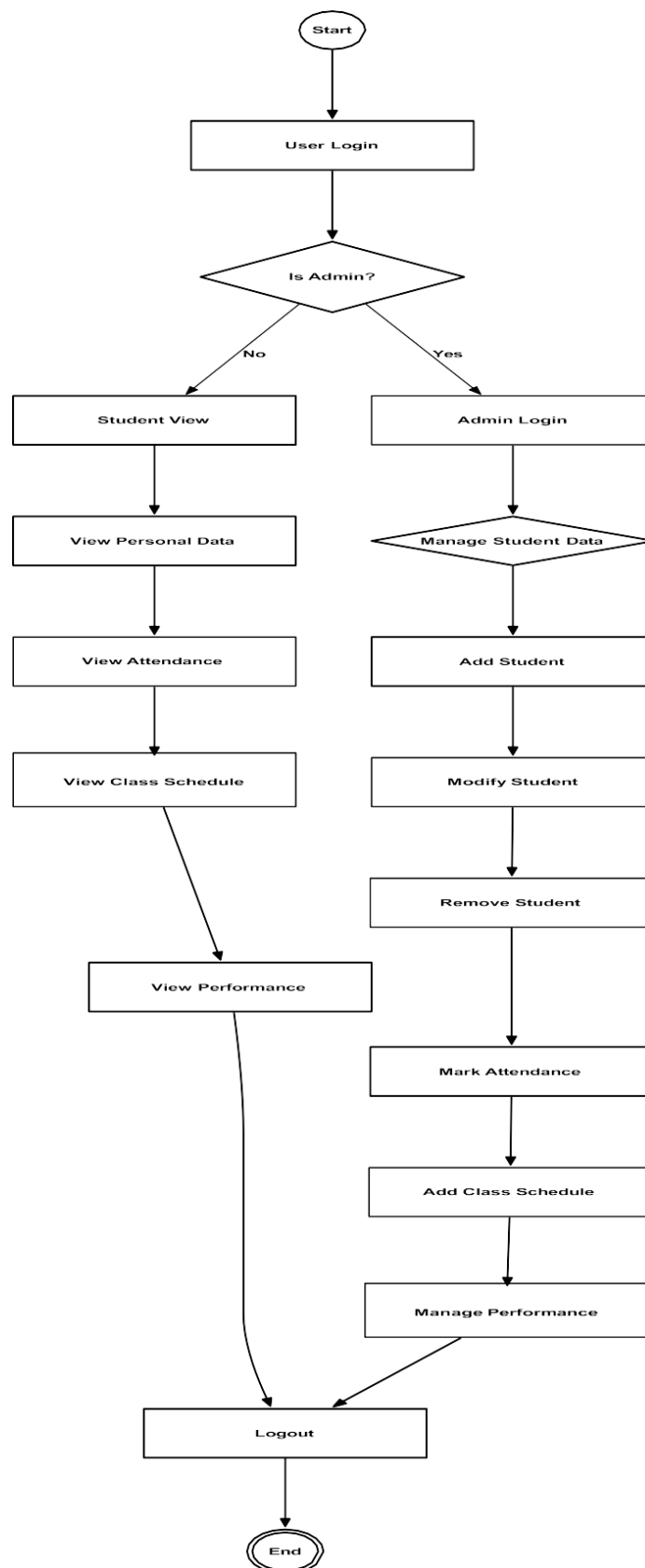
3.6.3 Class Diagram :-



3.6.4 Sequence Diagram :-



3.6.5 Activity Diagram :-



CHAPTER 4

System Design

4.1 Basic Modules

The **MyEducationPortal** system is designed with two main modules: **Admin (Management)** and **Student (User)**. Each module is responsible for specific tasks that together create an efficient and organized class management system.

1. Admin (Management) Module

This module allows authorized management personnel or faculty to manage all essential academic operations.

Key Functionalities:

1. Student Management:

- Add new student records with necessary details (name, ID, class, contact information).
- Edit or update existing student information.
- Remove student data for inactive or graduated students.

2. Attendance Management:

- Mark daily attendance for classes.
- View and modify attendance records as needed.
- Generate attendance reports for analysis.

3. Schedule Management:

- Create and update class schedules.
- Assign schedules to specific students or classes.
- Manage schedule changes dynamically.

4. Performance Management:

- Enter and update student grades or marks.
- Generate performance reports for students.
- Monitor academic progress over time.

5. Secure Login:

- Provides a secure authentication system for management users.
- Restricts unauthorized access to sensitive data.

2. Student (User) Module

This module is designed to offer students a transparent and simple interface to access their academic information.

Key Functionalities:

1. Profile Viewing:

- View personal data (name, enrollment number, class details, etc.).

2. Attendance Monitoring:

- Check attendance records and percentages.
- View monthly or semester-wise attendance summaries.

3. Class Schedule Viewing:

- Access updated class and lecture schedules.
- Stay informed about changes in timetable.

4. Academic Performance:

- View marks, grades, or progress reports.
- Monitor academic progress regularly.

5. Secure Login:

- Individual login credentials for each student.
- Ensures that only authorized students can view their data.

3. Authentication & Database Module

Although not directly visible to users, this module plays a crucial role in maintaining system security and data consistency.

Key Functionalities:

1. User Authentication:

- Separate login systems for students and admin.
- Secure password handling and session management.

2. Database Operations:

- Store all data (students, schedules, attendance, performance) in a centralized MySQL database.
- Enable data retrieval and update operations with high efficiency.

4.2 Data Design

4.2.1 Schema Design:

Table Name	Field	Type	Key Type	Description
Admin	Admin_id	Integer	Primary Key	Unique admin identifier
	Username	String(50)	Unique	Admin login username
	Password	String(100)	—	Admin login password (hashed recommended)
	Role	String(20)	Enum('Admin','Student')	Role type (e.g., Admin, Student)
Standard	Standard_id	Integer	Primary Key	Unique ID for each standard/class
	Standard_name	String(20)	Unique	Name of the class (e.g., "10th A")
	School_name	String(100)	—	School/branch name
Student	Student_id	Integer	Primary Key	Unique identifier for each student
	Roll_no	Integer	Unique	Roll number in class
	Student_name	String(80)	—	Full name of student
	Gender	String(10)	—	Gender of student
	DOB	Date	—	Date of Birth
	Admission_date	Date	—	Date of Admission
	Standard_id	Integer	Foreign Key	Refers to Standard.Standard_id
	Password	String(100)	—	Student login password

Table Name	Field	Type	Key Type	Description
Schedule	Schedule_id	Integer	Primary Key	Unique identifier for schedule entry
	Date	Date	—	Schedule date
	Subject	String(50)	—	Subject name
	Start_time	Time	—	Lecture start time
	End_time	Time	—	Lecture end time
	Professor_name	String(100)	—	Assigned professor
	Standard_id	Integer	Foreign Key	Refers to Standard.Standard_id
Attendance	Attendance_id	Integer	Primary Key	Unique identifier for attendance record
	Student_id	Integer	Foreign Key	Refers to Student.Student_id
	Schedule_id	Integer	Foreign Key	Refers to Schedule.Schedule_id
	Date	Date	—	Date of lecture
Performance	Performance_id	Integer	Primary Key	Unique identifier for performance record
	Student_id	Integer	Foreign Key	Refers to Student.Student_id
	Subject	String(50)	—	Subject name
	Marks	Integer	—	Marks scored in exam/test
	Exam_date	Date	—	Date of exam
	Attendance_percentage	Float	—	Attendance percentage of student
	Overall_grade	String(5)	—	Grade based on marks + attendance
	Remark	String(100)	—	Teacher's remark

4.2.2 Data Integrity and Constraints

To ensure data consistency, reliability, and security, the following constraints and integrity rules are applied:

1. Primary Key Constraints:

- Each table has a primary key (e.g., `student_id`, `attendance_id`) to ensure unique identification of records.

2. Foreign Key Constraints:

- Relationships are established between tables using foreign keys (e.g., `student_id` in Attendance and Performance tables referencing Students table).

3. Not Null Constraints:

- Essential fields such as student name, `enrollment_no`, and date fields cannot be null to avoid incomplete records.

4. Unique Constraints:

- Enrollment numbers and admin usernames must be unique.

5. Data Type Constraints:

- Appropriate data types (INT, VARCHAR, DATE, ENUM) are assigned to ensure data validity.

6. Referential Integrity:

- Cascading updates/deletes applied where necessary to maintain consistent data relationships.

7. Authentication and Security Constraints:

- Passwords are stored in encrypted form.
- Restricted access levels for Admin and Student roles.

4.3 Procedural Design

Procedural design provides a systematic approach for developing the algorithms and processes of *My Education Portal*. It defines the flow of operations, the data structures involved, and the logic required to achieve the system's functionalities.

4.3.1 Logic Diagrams

The following diagrams represent the systematic flow of procedures in the system:

1. Login Flow (Admin/Student):

- User enters username and password → System verifies credentials → If valid → Redirect to respective dashboard (Admin/Student). Else → Show error message.

(Control Flow: Login → Authentication → Role Identification → Redirect)

2. Attendance Management (Admin):

- Admin logs in → Selects class → Marks student attendance (Present/Absent) → Data stored in Attendance Table → Generate daily/weekly/monthly report.

3. Performance Management (Admin):

- Admin logs in → Selects student → Enters subject marks → Data stored in Performance Table → Reports available to students/parents.

4. Student Dashboard Flow:

- Student logs in → Dashboard displays profile, attendance summary, class schedule, performance report.

4.3.2 Data Structures

The following data structures are utilized in the procedures, aligned with the schema design:

- **Students Table:**
{ student_id, name, class, contact_info, enrollment_no }
- **Attendance Table:**
{ attendance_id, student_id, date, status }
- **Schedule Table:**
{ schedule_id, class, subject, date_time }
- **Performance Table:**
{ performance_id, student_id, subject, marks }
- **Admin Table:**
{ admin_id, username, password }
- **Login (optional):**
{ user_id, username, password, role }

4.3.3 Algorithm Design

(a) Login Algorithm

Input: username, password

Output: User redirected to Admin/Student dashboard or error message

Steps:

1. Accept username and password.
2. Query Login table for matching credentials.
3. If match found → Identify role.
 - If role = Admin → Redirect to Admin dashboard.
 - If role = Student → Redirect to Student dashboard.
4. Else → Display “Invalid Login”.

(b) Attendance Recording Algorithm (Admin)

Input: Class, student list, attendance status

Output: Attendance records stored in database

Steps:

1. Admin selects class.
2. For each student in the class:
 - Mark as Present/Absent.
 - Insert record into Attendance Table {attendance_id, student_id, date, status}.
3. Confirm save.
4. Generate summary report if required.

(c) Performance Recording Algorithm (Admin)

Input: Student ID, subject, marks

Output: Performance record updated in database

Steps:

1. Admin selects student and subject.
2. Enter marks/grade.
3. Insert/Update record in Performance Table {performance_id, student_id, subject, marks}.
4. Allow students to view updated reports on login.

(d) Student Dashboard Algorithm

Input: Student ID (after login)

Output: Display profile, attendance, schedule, and performance

Steps:

1. Query Students Table for profile details.
2. Query Attendance Table for attendance summary.
3. Query Schedule Table for class timetable.
4. Query Performance Table for marks and progress.
5. Render all information in a unified dashboard.

4.4 User Interface Design

The User Interface (UI) of *My Education Portal* is designed to ensure simplicity, accessibility, and efficiency for both students and administrators. It focuses on providing role- based functionalities with a clear, responsive, and intuitive design.

4.4.1 User, Task, and Environment Analysis

Users

1. Admin/Management:

- Manage student data (add, update, delete records).
- Enter and monitor attendance and performance.
- Create/manage schedules.
- Generate reports.

2. Students:

- View attendance records and summaries.
- Access class schedules.
- Check academic performance.
- View personal profile.

Tasks

- **Admin Tasks:** Login, student management, attendance tracking, performance entry, schedule updates, report generation.
- **Student Tasks:** Login, view profile, monitor attendance, access timetable, view performance reports.

Environment

- **Platform:** Web-based system (accessible via browser).
- **Devices:** Desktop, laptop, tablet, and mobile.
- **Constraints:** Requires stable internet, responsive design for multiple screen sizes.

4.4.2 Mapping Requirements into User Interface

- **Login Page:** Secure role-based login.
- **Admin Dashboard:** Quick access to student records, attendance, performance, and schedules.
- **Student Dashboard:** Personalized view of profile, attendance, performance, and schedule.
- **Navigation:** Sidebar or top navigation bar for accessing modules easily.
- **Reports/Charts:** Graphical representation of performance and attendance for better understanding.

4.4.3 Architecture of User Interface

The UI architecture is structured in **two layers**:

1. External Components (Visible to Users):

- Login Screen
- Navigation Menus
- Dashboards (Admin/Student)
- Forms (Attendance entry, Marks entry, Schedule update)
- Reports (tabular + graphical charts)

2. Internal Components (Supporting the UI):

- Flask template rendering (Jinja2) for dynamic data presentation.
- Form validation (JavaScript + Flask).
- Secure session management.
- Database connectivity with MySQL to fetch/store records.

4.5 Test Cases Design

Test Case ID	Module	Test Case Description	Pre-Condition	Test Steps	Expected Result	Actual Result	Status (Pass/Fail)
TC01	Login	Verify student login with valid credentials	Student registered in system	1. Open login page 2. Enter valid username & password 3. Click "Login"	Student redirected to Student Dashboard		
TC02	Login	Verify login with invalid credentials	User enters wrong data	1. Enter invalid username/password 2. Click "Login"	System shows error message "Invalid credentials"		
TC03	Student Profile	Check if student profile loads correctly	Student logged in	1. Login as student 2. Navigate to profile section	Profile details displayed correctly		
TC04	Attendance	Verify student can view attendance records	Student has attendance data	1. Login as student 2. Go to "View Attendance"	Attendance records are displayed with correct dates & status		
TC05	Performance	Verify performance report	Student has marks uploaded	1. Login as student 2. Click "Performance"	Marks displayed subject-wise		
TC06	Schedule	Verify class schedule is visible to student	Schedule exists for class	1. Login as student 2. Open "Schedule" tab	Class schedule displayed properly		
TC07	Admin – Manage Students	Verify admin can add student	Admin logged in	1. Login as admin 2. Go to "Add Student" 3. Enter details & save	Student record successfully added to DB		
TC08	Admin – Manage Students	Verify admin can delete student	Student exists	1. Login as admin 2. Select student 3. Click "Delete"	Student record removed from system		
TC09	Admin – Attendance	Verify admin can mark attendance	Student exists in class	1. Login as admin 2. Select class 3. Mark attendance 4. Save	Attendance record saved for selected date		

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Implementation Approaches

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>MyEducationPortal - Login</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.3/font/bootstrap-
icons.min.css">
</head>
<body>
  <div class="login-card">
    <div class="avatar">
      <i class="bi bi-person-circle"></i>
    </div>
    <h2>Sign In</h2>
    {% with messages = get_flashed_messages() %}
    {% if messages %}
      <div class="error">{{ messages[0] }}</div>
    {% endif %}
    {% endwith %}
    <form method="POST" autocomplete="off">
      <input type="text" name="username" class="form-control" placeholder="Username" required autofocus
autocomplete="off">
      <input type="password" name="password" class="form-control" placeholder="Password" required
autocomplete="off">
      <!-- Role will be auto-detected: only username & password required -->
      <button type="submit" class="login-btn mt-2">Login <i class="bi bi-box-arrow-in-right ms-
1"></i></button>
    </form>
  </div>
</body>
```

</html>

Login

```
@app.route('/login', methods=['GET', 'POST'])
```

```
def login():
```

```
    if request.method == 'POST':
```

```
        username = request.form.get('username')
```

```
        password = request.form.get('password')
```

```
        # Auto-detect role and authenticate.
```

```
        # 1) Root (credentials stored in config.json)
```

```
    if username == parameters.get('Admin_user') and password == parameters.get('Admin_password'):
```

```
        session['user'] = username
```

```
        session['role'] = 'root'
```

```
        # Upsert a Login row for root so we can record last_login (store hashed for root)
```

```
        try:
```

```
            lr = Login.query.filter_by(account_id=username).first()
```

```
            now = datetime.datetime.now()
```

```
            if lr:
```

```
                lr.last_login = now
```

```
                lr.password = generate_password_hash(password)
```

```
                lr.role = 'root'
```

```
                db.session.add(lr)
```

```
            else:
```

```
                hashed = generate_password_hash(password)
```

```
                new_lr = Login(account_id=username, password=hashed, role='root', last_login=now)
```

```
                db.session.add(new_lr)
```

```
            db.session.commit()
```

```
        except Exception as e:
```

```
            db.session.rollback()
```

```
            print('Failed to upsert root login row:', e)
```

```
        return redirect(url_for('root_dashboard'))
```

```
    # 2) Try centralized Login table first (supports hashed or plaintext stored there)
```

```
    login_row = Login.query.filter_by(account_id=username).first()
```

```

if login_row:
    authenticated = False

    try:
        # support plaintext match or hashed password
        if login_row.password == password:
            authenticated = True
        else:
            authenticated = check_password_hash(login_row.password, password)
    except Exception:
        authenticated = False

if authenticated:
    session['user'] = login_row.account_id
    session['role'] = login_row.role
    try:
        login_row.last_login = datetime.datetime.now()
        db.session.add(login_row)
        db.session.commit()
    except Exception as e:
        db.session.rollback()
        print('Failed to update last_login for', login_row.account_id, e)

    if login_row.role == 'admin':
        return redirect(url_for('admin_profile', admin_id=login_row.account_id))
    elif login_row.role == 'student':
        return redirect(url_for('student_notifications'))
    elif login_row.role == 'root':
        return redirect(url_for('root_dashboard'))

# 3) If no Login row or password didn't match, try Admin table directly
admin = Admin.query.filter_by(Admin_id=username).first()

if admin and getattr(admin, 'Password', None) == password:
    # create/update Login row for admin with hashed password

```

```
try:
```

```
    now = datetime.datetime.now()
```

```
    hashed = generate_password_hash(password)
```

```
    lr = Login.query.filter_by(account_id=admin.Admin_id).first()
```

```
    if lr:
```

```
        lr.password = hashed
```

```
        lr.role = 'admin'
```

```
        lr.last_login = now
```

```
        db.session.add(lr)
```

```
    else:
```

```
        new_lr = Login(account_id=admin.Admin_id, password=hashed, role='admin', last_login=now)
```

```
        db.session.add(new_lr)
```

```
        db.session.commit()
```

```
except Exception as e:
```

```
    db.session.rollback()
```

```
    print('Failed to upsert admin login row:', e)
```

```
session['user'] = admin.Admin_id
```

```
session['role'] = 'admin'
```

```
return redirect(url_for('admin_profile', admin_id=admin.Admin_id))
```

```
# 4) Try Student table
```

```
student = Student_reg.query.filter_by(Student_id=username).first()
```

```
if student and getattr(student, 'Student_password', None) == password:
```

```
    # create/update Login row for student with hashed password (store plain password in Student_reg  
    remains for display)
```

```
    try:
```

```
        now = datetime.datetime.now()
```

```
        hashed = generate_password_hash(password)
```

```
        lr = Login.query.filter_by(account_id=student.Student_id).first()
```

```
        if lr:
```

```
            lr.password = hashed
```

```
            lr.role = 'student'
```



```

        lr.last_login = now
        db.session.add(lr)
    else:
        new_lr = Login(account_id=student.Student_id, password=hashed, role='student',
last_login=now)
        db.session.add(new_lr)
        db.session.commit()
except Exception as e:
    db.session.rollback()
    print('Failed to upsert student login row:', e)

session['user'] = student.Student_id
session['role'] = 'student'
return redirect(url_for('student_dashboard'))

# If nothing matched, show invalid credentials
flash('Invalid credentials. Please try again.', 'danger')
return render_template("login.html", parameters=parameters)

```

logout

```

@app.route("/logout")
def logout():
    """Clear session and redirect to login."""
    session.clear()
    flash('You have been logged out.', 'info')
    return redirect(url_for('login'))

```

Registration

```
@app.route('/Student_reg', methods=['GET', 'POST'])
@login_required
def register():
    # Prepare select lists for template
    standards = ['1st','2nd','3rd','4th','5th','6th','7th','8th','9th','10th','11th','12th']
    streams = ['Science','Commerce','Arts']

    # Ensure default batches exist for 11th/12th (admin/root can modify later via DB)
    try:
        for std in ['11th','12th']:
            for b in ['Entrance','Regular']:
                if not Batch.query.filter_by(Standard=std, Stream='Science', name=b).first():
                    db.session.add(Batch(name=b, Standard=std, Stream='Science'))
            for b in ['SP','Maths']:
                if not Batch.query.filter_by(Standard=std, Stream='Commerce', name=b).first():
                    db.session.add(Batch(name=b, Standard=std, Stream='Commerce'))
        db.session.commit()
    except Exception:
        db.session.rollback()

    # Fetch all batches for client-side filtering and convert to serializable dicts
    raw_batches = Batch.query.order_by(Batch.Standard, Batch.Stream, Batch.name).all()
    all_batches = []
    for b in raw_batches:
        try:
            all_batches.append({
                'id': getattr(b, 'id', None),
                'name': getattr(b, 'name', None),
                'Standard': getattr(b, 'Standard', None),
                'Stream': getattr(b, 'Stream', None),
            })
        except Exception:
```

```
# Skip any non-serializable entries
```

```
continue
```

```
if request.method == "POST":
```

```
    Student_name = request.form.get("student_name")
```

```
    print(f'message object of student name \n \n : {format(request.form)}')
```

```
    Gender = request.form.get("gender")
```

```
    DOB = request.form.get("dob")
```

```
    School_name = request.form.get("school_name")
```

```
    Standard = request.form.get("standard")
```

```
    Stream = request.form.get("stream")
```

```
    Student_batch = request.form.get("batch")
```

```
    # If user provided a batch name via textbox, ensure Batch row exists
```

```
    try:
```

```
        if Student_batch and Student_batch.strip():
```

```
            sb_name = Student_batch.strip()
```

```
            # normalize stream value for lookup
```

```
            sb_stream = Stream.strip() if Stream else None
```

```
            existing_batch = Batch.query.filter_by(name=sb_name, Standard=Standard,  
Stream=sb_stream).first()
```

```
            if not existing_batch:
```

```
                new_batch = Batch(name=sb_name, Standard=Standard, Stream=sb_stream)
```

```
                db.session.add(new_batch)
```

```
                # don't commit now; will commit together with student record
```

```
    except Exception:
```

```
        db.session.rollback()
```

```
    Address = request.form.get("address")
```

```
    Father_name = request.form.get("father_name")
```

```
    Father_profession = request.form.get("father_profession")
```

```
    Mother_name = request.form.get("mother_name")
```

```
    Mother_profession = request.form.get("mother_profession")
```

```
    Student_number = request.form.get("student_number")
```

```
    Parent_number = request.form.get("parent_number")
```

```

# Server-side validation

errors = []

# Required checks

if not Student_name or not Student_name.strip():
    errors.append('Student name is required.')

if not School_name or not School_name.strip():
    errors.append('School/College name is required.')

if not Standard or not Standard.strip():
    errors.append('Standard/Batch is required.')

if not Student_number or not str(Student_number).strip():
    errors.append('Student phone/number is required.')

if not Parent_number or not str(Parent_number).strip():
    errors.append('Parent phone number is required.')

if not Gender:
    errors.append('Gender is required.')

if not DOB:
    errors.append('Date of birth is required.')


# Validate numeric phone fields

try:
    if Student_number and not str(Student_number).isdigit():
        errors.append('Student phone/number must contain only digits.')
except Exception:
    errors.append('Invalid student number.')

try:
    if Parent_number and not str(Parent_number).isdigit():
        errors.append('Parent phone number must contain only digits.')
except Exception:
    errors.append('Invalid parent number.')


# Validate DOB format (expecting YYYY-MM-DD)

try:

```

if DOB:

```
datetime.datetime.strptime(DOB, '%Y-%m-%d')
```

except Exception:

```
errors.append('Date of birth must be in YYYY-MM-DD format.')
```

Uniqueness checks

try:

```
# Student phone/number must be unique across student records.
```

```
# Parent numbers are intentionally NOT checked against existing student numbers
```

```
# (parents may share numbers or reuse a student's number).
```

```
if Student_number and Student_reg.query.filter_by(Student_number=Student_number).first():
```

```
    errors.append('Student phone/number is already registered to another student. Student phone must be unique.')
```

except Exception:

```
# ignore DB errors here; will be caught later
```

```
pass
```

Validate uploaded file extension

```
photo_file = request.files.get('photo')
```

```
if photo_file and photo_file.filename:
```

```
    if not allowed_file(photo_file.filename):
```

```
        errors.append('Uploaded photo must be an image (png/jpg/jpeg/gif).')
```

if errors:

```
# Re-render form with previous inputs and error list displayed at top
```

```
    return render_template('Student_reg.html', parameters=parameters, form=request.form, errors=errors, standards=standards, streams=streams, batches=all_batches)
```

1. Handle Uploaded File

```
photo_file = request.files.get("photo")
```

```
filename = None
```

```
if photo_file and allowed_file(photo_file.filename):
```

```
    filename = secure_filename(photo_file.filename)
```

```
photo_file.save(os.path.join(app.config["UPLOAD_FOLDER"], filename))
```

2. Handle Captured Photo (Base64)

```
if not filename and request.form.get("captured_image"):
```

```
    image_data = request.form["captured_image"]
```

```
    image_data = image_data.replace("data:image/png;base64,", "")
```

```
    img_bytes = base64.b64decode(image_data)
```

```
    filename = f'{Student_name.replace(' ', '_')}.png'
```

```
    with open(os.path.join(app.config["UPLOAD_FOLDER"], filename), "wb") as f:
```

```
        f.write(img_bytes)
```

```
# Ensure Student_photo is never None to satisfy NOT NULL DB constraint
```

```
if not filename:
```

```
    filename = 'placeholder.png'
```

```
# Create the Student_reg entry with a temporary unique Student_id so the INSERT succeeds
```

```
DOB = datetime.datetime.strptime(DOB, "%Y-%m-%d")
```

```
doa_store = datetime.date.today()
```

```
numbers = random.sample(range(1, 11), 4)
```

```
numbers = ".join(str(n) for n in numbers)
```

```
Student_id = Student_name[:4]+doa_store.strftime('%Y')+numbers
```

```
Student_pass= Student_name[:4]+DOB.strftime('%Y')
```

```
entry = Student_reg(
```

```
    Student_name=Student_name,
```

```
    Gender=Gender,
```

```
    DOB=DOB,
```

```
    School_name=School_name,
```

```
    Standard=Standard,
```

```
    Student_batch=Student_batch,
```

```
    Stream=Stream,
```

```
    Address=Address,
```

```
    Father_name=Father_name,
```

```
    Father_profession=Father_profession,
```

```
    Mother_name=Mother_name,
```

```

    Mother_profession=Mother_profession,
    Student_number=Student_number,
    Parent_number=Parent_number,
    Student_photo=filename,
    Student_id=Student_id,
    Student_password=Student_pass,
    DOA=doa_store,
    Added_by=session.get('user'),
    Created_at=datetime.datetime.now()
)

db.session.add(entry)

try:
    # Flush to get Sr_no assigned without committing yet
    db.session.flush()
except Exception as flush_e:
    # If flush fails, rollback and report
    db.session.rollback()
    print('Failed to insert student entry:', flush_e)
    flash('Failed to register student. Please try again.', 'danger')

    return render_template('Student_reg.html', parameters=parameters, standards=standards,
streams=streams, batches=all_batches)

# Now Sr_no should be available — finalize Student_id and password based on Sr_no
try:
    sr = entry.Sr_no
    # Build name part (first 4 alphanumeric characters, uppercase)
    cleaned = "".join([c for c in (Student_name or "") if c.isalnum()])
    name_part = (cleaned[:4].upper()).ljust(4, 'X')
    # Year from DOA (DOA stored in doa_store)
    try:
        year = doa_store.year
    except Exception:

```

```

year = datetime.date.today().year

generated_id = f'{name_part}{year}{sr:04d}'

# Generate password: first 4 chars of name + 4-digit random number
rand_num = f'{secrets.randbelow(10000):04d}'
generated_password = f'{name_part}{rand_num}'

# Update the entry with final Student_id and password
entry.Student_id = generated_id
entry.Student_password = generated_password
db.session.add(entry)

# Commit student record (credentials stored in Student_reg.Student_password)
try:
    db.session.commit()

    # After successfully committing the student record, create a Login entry
    try:
        hashed = generate_password_hash(generated_password)
        now = datetime.datetime.now()
        lr = Login.query.filter_by(account_id=generated_id).first()
        if lr:
            lr.password = hashed
            lr.role = 'student'
            lr.last_login = now
            db.session.add(lr)
        else:
            new_lr = Login(account_id=generated_id, password=hashed, role='student', last_login=now)
            db.session.add(new_lr)
            db.session.commit()
    except Exception:
        db.session.rollback()

    # Prepare modal data to show registration success with credentials

```



```

modal_name = Student_name

modal_id = generated_id

modal_password = generated_password

flash(f'Student Registered. Student ID: {generated_id} Password: {generated_password}',
'success')

return render_template("Student_reg.html", parameters=parameters, generated_id=modal_id,
generated_password=modal_password, generated_name=modal_name, standards=standards,
streams=streams, batches=all_batches)

except Exception as le:

    db.session.rollback()

    print('Failed to commit student record:', le)

    flash(f'Student Registered but failed to save record. Student ID: {generated_id}', 'warning')
except Exception as e:

    db.session.rollback()

    print('Error generating Student_id/password:', e)

    flash('Student registered but failed to generate Student ID/password automatically.', 'warning')
except Exception as e:

    db.session.rollback()

    print('Error generating Student_id/password:', e)

    flash('Student registered but failed to generate Student ID/password automatically.', 'warning')

print("Student Registered:", Student_name, "| Photo Saved As:", filename)

return render_template("Student_reg.html", parameters=parameters, standards=standards,
streams=streams, batches=all_batches)

```

5.3 Testing Approach

5.3.1 Unit Testing

Unit testing focuses on testing individual modules independently to verify that each component performs its intended function correctly.

In this project, the following modules were tested separately:

Module	Testing Purpose
Login Module	Verify authentication functionality
Student Management	Test adding, editing, deleting students
Attendance Module	Verify attendance marking and storage
Performance Module	Ensure marks are stored correctly
Schedule Module	Verify class schedule display

Example:

Testing login module:

- Enter correct credentials → Login successful
- Enter incorrect credentials → Error message displayed

5.3.2 Integrated Testing

Integration testing involves combining all system modules and testing the complete application to verify that the modules interact correctly.

The following interactions were tested:

- Login → Dashboard redirection
- Admin adding students → Student data visible in system
- Attendance marking → Attendance visible to students
- Performance entry → Marks displayed in student dashboard

Integration testing ensured that:

- Data flows correctly between modules
- Database operations function correctly
- System behaves consistently across different pages

5.3.3 Beta Testing

Beta testing was performed by allowing a limited number of users to test the system in a real-world environment.

Participants included:

- Students
- Administrative staff

Users were asked to:

- Log into the portal
- View attendance and schedule
- Check performance reports

Feedback was collected regarding:

- User interface usability
- System responsiveness
- Ease of navigation

Based on user feedback, minor adjustments were made to improve the interface and system performance.

5.4 Modifications and Improvements

During the testing phase, several modifications were implemented to improve the functionality and performance of the system.

Improvements implemented include:

1. **Login Validation Improvement**
 - Added error messages for invalid login attempts.
2. **Database Structure Optimization**
 - Improved table relationships using foreign keys.
3. **User Interface Improvements**
 - Improved dashboard layout for better readability.
4. **Bug Fixes**
 - Corrected issues related to incorrect data retrieval.
5. **Performance Optimization**
 - Optimized database queries to reduce loading time.

These modifications improved system stability and enhanced user experience.

5.5 Test Cases

The following table summarizes important test cases used during system testing.

Test Case ID	Module	Test Case Description	Pre-Condition	Test Steps	Expected Result	Actual Result	Status
TC01	Login	Verify student login with valid credentials	Student registered in system	1. Open login page 2. Enter valid username & password 3. Click "Login"	Student redirected to Student Dashboard	Student successfully redirected to dashboard after login	Pass
TC02	Login	Verify login with invalid credentials	User enters wrong data	1. Enter invalid username/password 2. Click "Login"	System shows error message "Invalid credentials"	Error message displayed and login denied	Pass
TC03	Student Profile	Check if student profile loads correctly	Student logged in	1. Login as student 2. Navigate to profile section	Profile details displayed correctly	Student profile information displayed properly	Pass
TC04	Attendance	Verify student can view attendance records	Student has attendance data	1. Login as student 2. Go to "View Attendance"	Attendance records are displayed with correct dates & status	Attendance data retrieved successfully from database	Pass
TC05	Performance	Verify performance report	Student has marks uploaded	1. Login as student 2. Click "Performance"	Marks displayed subject-wise	Subject-wise marks displayed correctly on dashboard	Pass
TC06	Schedule	Verify class schedule is visible to student	Schedule exists for class	1. Login as student 2. Open "Schedule" tab	Class schedule displayed properly	Schedule displayed correctly with subject and time	Pass
TC07	Admin – Manage Students	Verify admin can add student	Admin logged in	1. Login as admin 2. Go to "Add Student" 3. Enter details & save	Student record successfully added to DB	Student information stored successfully in Student table	Pass

Test Case ID	Module	Test Case Description	Pre-Condition	Test Steps	Expected Result	Actual Result	Status
TC08	Admin – Manage Students	Verify admin can delete student	Student exists	1. Login as admin 2. Select student 3. Click "Delete"	Student record removed from system	Student record deleted from database successfully	Pass
TC09	Admin – Attendance	Verify admin can mark attendance	Student exists in class	1. Login as admin 2. Select class 3. Mark attendance 4. Save	Attendance record saved for selected date	Attendance stored in Attendance table successfully	Pass
TC10	Admin – Performance	Verify admin can add marks for student	Student exists and admin logged in	1. Login as admin 2. Select student 3. Enter marks 4. Save	Marks stored successfully in database	Marks saved in Performance table successfully	Pass
TC11	Database	Verify data retrieval from database	Data stored in tables	1. Query student records 2. Display on dashboard	System retrieves correct data from database	Data fetched successfully without errors	Pass
TC12	Session Management	Verify logout functionality	User logged in	1. Click logout button	User session ends and redirected to login page	User redirected to login page and session cleared	Pass

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Test Reports

Testing was conducted to verify the correctness, reliability, and efficiency of the My Education Portal system. The purpose of testing was to ensure that all modules perform their intended functions and that the system behaves correctly under different conditions.

Testing involved executing the test cases described in Chapter 5 using various input values and observing the resulting outputs.

Login Module Testing

The login functionality was tested using both valid and invalid user credentials.

Sample Input (Valid Credentials)

Username: student01

Password: correct password

Output:

The system successfully authenticates the user and redirects them to the **Student Dashboard**.

Sample Input (Invalid Credentials)

Username: student01

Password: incorrect password

Output:

The system displays an error message:

“Invalid username or password.”

Result:

The login module correctly validates user credentials and prevents unauthorized access.

Student Profile Testing

The student profile module was tested to verify that students can view their personal information after logging into the system.

Sample Input:

Student logs into the system and opens the profile section.

Output:

The system displays the student's details including name, roll number, class, and other stored information.

Result:

The profile module retrieves and displays the correct information from the database.

Attendance Module Testing

The attendance module was tested by marking attendance through the admin panel and verifying that the attendance records are displayed correctly.

Sample Input:

Admin selects a class and marks attendance for students.

Output:

Attendance data is stored in the database and displayed when students view their attendance records.

Result:

Attendance records are saved successfully and retrieved correctly from the database.

Performance Module Testing

The performance module was tested by entering marks for students and verifying that the marks are displayed in the student dashboard.

Sample Input:

Admin enters subject marks for a student.

Output:

The marks are stored in the database and displayed in the performance report section.

Result:

The performance module successfully records and displays student marks.

Schedule Module Testing

The schedule module was tested to ensure that class schedules created by the admin are visible to students.

Sample Input:

Admin creates a class schedule.

Output:

Students can view the schedule in their dashboard.

Result:

The schedule information is displayed correctly for the students.

Overall Testing Results

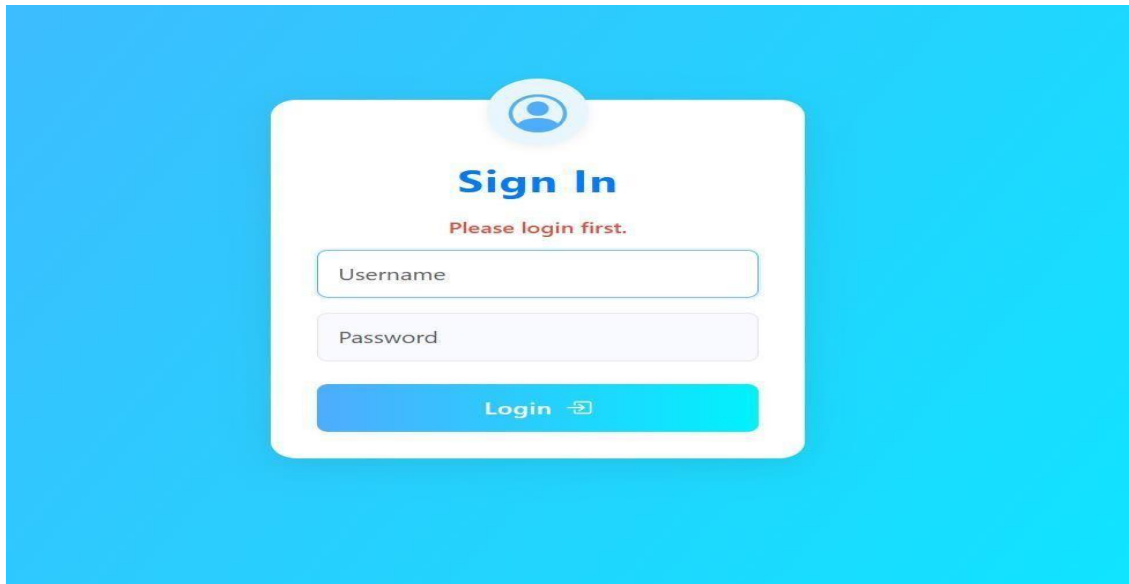
The test reports indicate that all system modules are functioning correctly. The system successfully handles:

- User authentication
- Student data management
- Attendance tracking
- Academic performance monitoring
- Schedule management

The results confirm that the system performs accurately and meets the functional requirements defined in earlier chapters.

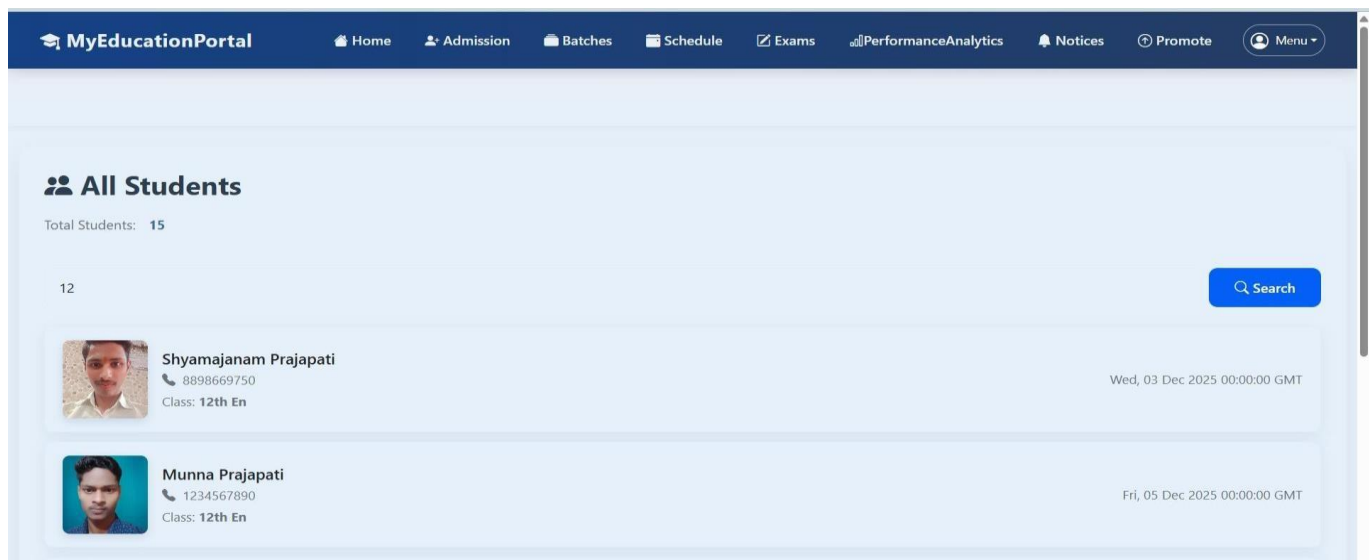
6.2 User Documentation

Login Page



A login page with a bright blue background. In the center is a white rounded rectangle containing a 'Sign In' form. At the top of the form is a circular icon with a person silhouette. Below it, the text 'Sign In' is in bold blue. Underneath is a red error message: 'Please login first.' The form has two input fields: 'Username' and 'Password'. At the bottom is a blue 'Login' button with a right-pointing arrow icon.

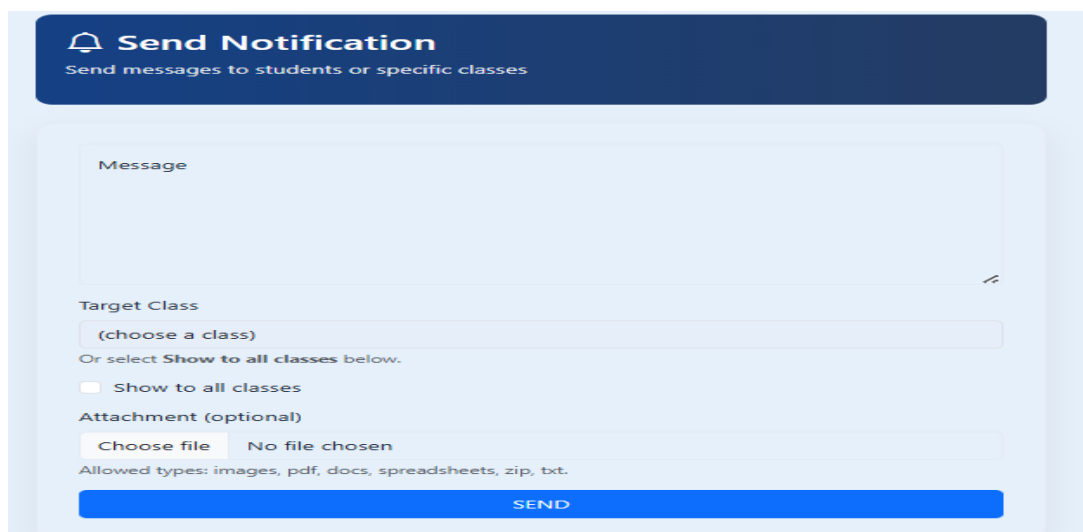
Admin Dashboard



An admin dashboard for 'MyEducationPortal'. The top navigation bar is dark blue with links: Home, Admission, Batches, Schedule, Exams, PerformanceAnalytics, Notices, Promote, and a Menu dropdown. The main content area is light blue. It starts with a section 'All Students' showing 'Total Students: 15'. Below this is a table with student information. The first student is Shyamajanam Prajapati, with phone number 8898669750, class 12th En, and a last update of 'Wed, 03 Dec 2025 00:00:00 GMT'. The second student is Munna Prajapati, with phone number 1234567890, class 12th En, and a last update of 'Fri, 05 Dec 2025 00:00:00 GMT'. A search bar is on the right.

Student Name	Phone Number	Class	Last Update
Shyamajanam Prajapati	8898669750	Class: 12th En	Wed, 03 Dec 2025 00:00:00 GMT
Munna Prajapati	1234567890	Class: 12th En	Fri, 05 Dec 2025 00:00:00 GMT

Notice



A 'Send Notification' form with a dark blue header. The header contains a bell icon and the text 'Send messages to students or specific classes'. The form has a 'Message' text area, a 'Target Class' dropdown menu (currently showing '(choose a class)'), and a checkbox for 'Show to all classes'. Below these is an 'Attachment (optional)' section with a 'Choose file' button and the text 'No file chosen'. At the bottom is a blue 'SEND' button. Allowed types for attachments are images, pdf, docs, spreadsheets, zip, and txt.

Registration Form

PHOTO

Upload Student Photo *

Max Upload Size: 2MB

Choose file

No file chosen

Or Capture Photo *

Open Camera

Capture

School/College Name *

Standard *

Select Standard

Student Name *

Student Phone Number *

Parent Phone Number *

Gender *

Male

Female

Other

Date of Birth *

dd-mm-yyyy

Address *

Father Name *

Father Profession *

Mother Name *

Mother Profession *

Register Student

Performance Analysis

Performance Analytics

Overview of all classes and their academic performance

Total Classes

4

Total Students

15

Total Exams

1

Overall Avg

33.33%

Class Performance Overview

Class/Standard	Students	Exams Conducted	% Class Average	Action
10th	9	1	33.33%	View Details
12th	3	0	No data	View Details
12th En	2	0	No data	View Details
9th	1	0	No data	View Details

Page | 67

Student Dashboard

Student Dashboard

Welcome back, Ashish Jaiswar | Class: 10th | ID: ASHI20250004

Dashboard

Performance

Attendance

Schedule

Notices

Menu

Total Classes

8

Present

7

Absent

1

% Attendance

87.5%

Recent Attendance

Date	Subject	Status
2026-02-13	English	Present
2026-01-20	English	Present
2026-01-11	History	Present
2026-01-04	maths	Present
2026-01-02	Geography	Present
2025-12-19	maths	Present
2025-12-08	Geography	Absent
2025-12-05	English	Present

View Full Attendance

Today's Schedule

No schedule for today.

Previous

All Schedules

Upcoming

Performance

Exam Date	Subject	Marks	Remarks
2025-12-06	maths	40.0 / 40	-

View Full Performance

Personal Information

Student ID:

ASHI20250004

Name:

Ashish Jaiswar

Class:

10th

School:

dayanand vidyalaya

Father:

ravindra jaiswar

Mother:

seema

Phone:

7718068532

Gender:

Male

View Full Profile

Quick Actions

View Attendance

View Schedule

Announcements

Logout

Attendance Management

MyEducationPortal

Home

Admission

Batches

Schedule

Exams

PerformanceAnalytics

Notices

Promote

Menu

Attendance — maths (Shyam sir)

Class: 10th · Date: 2026-03-15 · 10:30 AM - 12:30 PM

Back to Schedules

Today's Schedule: You can mark or edit attendance for today.

ID	Name	Attendance
	Ashish Jaiswar ID: ASHI20250004 · 7718068532	PresentAbsent Reason for absence
	Riya yadav ID: RIYA20250005 · 9321649469	PresentAbsent Reason for absence
	Riya Thale ID: RIYA20250006 · 9326563484	PresentAbsent Reason for absence
	Munna Prajapati ID: MUNN20250010 · 4578986514	PresentAbsent Reason for absence
	Nikhil Maurya	PresentAbsent

Student Performance Dashboard

Performance Dashboard

Ashish Jaiswar | 10th | Student ID: ASH20250004

Total Exams

1

Overall Percentage

100.0%

Your Rank

1

Class Strength

3

Performance Distribution

maths (100.0%)

Summary

Marks Summary

40.0/40.0

Highest: 40.0

Lowest: 40.0

Average: 40.0

All Exams & Marks

Date	Subject	Marks	Percentage	Remarks	Action
2025-12-06	maths	40.0 / 40	100.0%	-	View Details

Student Attendance Report

Student Dashboard

Welcome back, Ashish Jaiswar | Class: 10th | ID: ASH20250004

Dashboard

Performance

Attendance

Schedule

Notices

Menu

Attendance Records

Ashish Jaiswar | 10th | Student ID: ASH20250004 | Showing attendance from 2025-12-05 to 2026-02-13

Present: 7

Absent: 1

Date	Class	Subject	Professor	Start Time	End Time	Status	Remark
2026-02-13	10th	English	Akash Sir	01:30 PM	02:30 PM	Present	
2026-01-20	10th	English	Shyam sir	12:30 PM	03:30 PM	Present	
2026-01-11	10th	History	shyam	07:00	09:00	Present	
2026-01-04	10th	maths	Shyam sir	02:00	04:00	Present	
2026-01-02	10th	Geography	Supriya Ma'am	02:00	03:00	Present	
2025-12-19	10th	maths	Akanksha	01:30	02:30	Present	
2025-12-08	10th	Geography	Akash	12:00	01:00	Absent	
2025-12-05	10th	English	Akanksha	07:00	10:00	Present	

© 2025 My Education Portal. All rights reserved.

Schedule View

Student Dashboard

Welcome back, Ashish Jaiswar | Class: 10th | ID: ASH20250004

Dashboard

Performance

Attendance

Schedule

Notices

Menu

My Schedule

Viewing: Today

Previous

Today

Upcoming

Back to Dashboard

Date

2026-03-20

Subject

Biology

Time

01:45 PM - 03:00 PM

Professor

Akash Sir

Date

2026-03-20

Subject

Geography

Time

03:00 PM - 05:00 PM

Professor

Krishna Sir

Date

2026-03-20

Subject

maths

Time

10:00 AM - 12:00 PM

Professor

Shyam sir

Date

2026-03-20

Subject

English

Time

12:00 PM - 01:30 PM

Professor

Akanksha Ma'am

CHAPTER 7

CONCLUSIONS

7.1 Conclusion

The **My Education Portal – Class Management System** was developed to provide a centralized digital platform for managing classroom activities such as student records, attendance tracking, academic performance monitoring, and schedule management. The project was implemented using the **Flask framework for backend development, MySQL for database management, and HTML, CSS, and JavaScript for the frontend interface.**

The primary objective of the project was to design a web-based system that can simplify administrative tasks and provide students with easy access to their academic information. The system successfully achieves this objective by automating various classroom management operations that are traditionally handled manually.

Through the development of this project, several key aspects of modern web application development were implemented, including secure user authentication, database integration, dynamic content rendering, and role-based access control. The application allows administrators to manage student records efficiently while enabling students to monitor their attendance, schedules, and academic performance through a secure dashboard.

During the implementation and testing phases, the system was evaluated using multiple test cases to ensure that each module performs its intended function correctly. The testing results confirmed that the system handles different user interactions effectively and provides accurate outputs for operations such as login authentication, attendance recording, performance entry, and schedule management.

The development of this project also helped in understanding important software engineering concepts such as **system analysis, database design, procedural design, testing methodologies, and web application development using Python frameworks.** The integration of these concepts allowed the system to function reliably and meet the functional requirements defined in earlier chapters.

Overall, the **My Education Portal** project demonstrates how web technologies can be used to create an efficient digital solution for classroom management. The system reduces manual work, improves data accessibility, and enhances communication between administrators and students. The successful implementation of this project shows that such systems can significantly contribute to improving the efficiency of academic administration in educational institutions.

7.1.1 Significance of the System

The **My Education Portal system** holds significant importance in improving the management of classroom activities within educational institutions. Traditionally, academic information such as attendance records, student details, and performance reports were managed manually, which often resulted in errors, delays, and difficulties in accessing data.

The developed system provides several benefits, including:

1. Digital Transformation of Academic Processes

The system replaces manual record-keeping methods with a digital platform, reducing paperwork and improving the efficiency of academic management.

2. Improved Data Accessibility

Students and administrators can access academic information easily through a web-based interface from any device connected to the internet.

3. Increased Data Accuracy

Automated data storage and retrieval reduce the chances of human errors that commonly occur in manual systems.

4. Better Transparency

Students can monitor their attendance records, academic performance, and schedules directly, which improves transparency and accountability.

5. Efficient Administrative Management

Administrators can manage multiple student records and attendance data quickly, saving time and effort.

Overall, the system contributes to a more organized and efficient academic management environment.

7.2 Limitations of the System

Although the My Education Portal system successfully performs its intended functions, certain limitations were observed during the development and testing phases. These limitations mainly arise due to time constraints, resource limitations, and the scope of the project.

Some of the limitations of the system include:

1. Limited User Roles

The system currently supports only **administrator and student roles**. A separate teacher module could be implemented in future versions.

2. Local Deployment

The system is primarily designed to run on a local server during development. A full deployment on a cloud-based server was not implemented in this project.

3. Limited Notification Features

The current system does not include automated notifications such as email alerts or SMS notifications for attendance or performance updates.

4. No Mobile Application

The system is accessible through a web browser but does not include a dedicated mobile application.

5. Limited Data Analytics

Although the system displays basic information such as attendance and marks, advanced analytics and graphical reports are not included.

These limitations do not affect the core functionality of the system but indicate possible improvements for future development.

7.3 Future Scope of the Project

The My Education Portal system can be further enhanced by adding new features and improving the existing functionalities. Future developments can expand the system's capabilities and make it more suitable for real-world educational environments.

Possible future enhancements include:

1. Teacher Module Integration

A dedicated teacher module can be added to allow teachers to manage attendance, upload assignments, and update student performance directly.

2. Online Assignment and Exam System

The system can be extended to include features for online assignments, quizzes, and examinations.

3. Mobile Application Development

A mobile application version of the system can be developed to allow students and administrators to access the platform through smartphones.

4. Notification System

Integration of email or SMS notification services can notify students about attendance updates, schedule changes, or academic announcements.

5. Advanced Data Analytics

Graphical reports and analytics dashboards can be added to provide better insights into student performance and attendance trends.

6. Cloud Deployment

The system can be deployed on cloud platforms such as AWS, Azure, or Google Cloud to enable remote access for educational institutions.

7. Integration with Institutional Systems

The portal can be integrated with other systems such as library management, fee management, and learning management systems (LMS).

By implementing these enhancements, the My Education Portal can evolve into a comprehensive digital academic management system suitable for large educational institutions.

REFERENCES

- 1 Grinberg, M.** (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
- 2 Beazley, D., & Jones, B.** (2013). *Python Cookbook*. O'Reilly Media.
- 3 Silberschatz, A., Korth, H. F., & Sudarshan, S.** (2019). *Database System Concepts*. McGraw-Hill Education.
- 4 Ramakrishnan, R., & Gehrke, J.** (2003). *Database Management Systems*. McGraw-Hill.
- 5 Welling, L., & Thomson, L.** (2008). *PHP and MySQL Web Development*. Addison-Wesley (used as general reference for database-driven web systems).
- 6 Flask Official Documentation**
<https://flask.palletsprojects.com>
- 7 MySQL Official Documentation**
<https://dev.mysql.com/doc>
- 8 Python Official Documentation**
<https://docs.python.org>
- 9 Mozilla Developer Network (MDN) Web Docs – HTML, CSS, JavaScript**
<https://developer.mozilla.org>
- 10 Chart.js Documentation** (for dashboards/visualization reference)
<https://www.chartjs.org/docs>
- 11 GitHub Documentation – Version Control and Project Hosting**
<https://docs.github.com>
- 12 W3Schools Web Development Tutorials**
<https://www.w3schools.com>

GLOSSARY

Administrator (Admin)

A user with full access to the system who can manage student records, mark attendance, update academic performance, and manage schedules.

Authentication

The process of verifying the identity of a user before allowing access to the system using credentials such as username and password.

Backend

The server-side part of the application responsible for processing user requests, handling business logic, and communicating with the database. In this project, the backend is developed using the Flask framework.

Database

An organized collection of data stored electronically. In this project, MySQL is used to store student information, attendance records, schedules, and performance data.

Dashboard

A user interface that displays important system information such as attendance summaries, academic records, and schedules in an organized format.

Flask

A lightweight Python web framework used to develop the backend of the web application and handle routing, sessions, and server-side logic.

Frontend

The client-side part of the application that users interact with directly through the web browser. It includes HTML, CSS, and JavaScript components.

HTML (HyperText Markup Language)

The standard markup language used to create and structure web pages.

CSS (Cascading Style Sheets)

A stylesheet language used to design and style the layout and appearance of web pages.

JavaScript

A programming language used to create dynamic and interactive web page elements.

Jinja2

A template engine used in Flask to generate dynamic HTML pages by embedding Python variables into HTML templates.

Login System

A security feature that allows users to access the system by entering valid credentials.

MySQL

An open-source relational database management system used to store and manage structured data.

Schedule Management

A system feature that allows administrators to create and manage class schedules for students.

Student Module

A section of the system that allows students to view their profile, attendance records, class schedules, and academic performance.

APPENDICES

APPENDIX – A

Important Code Snippets

This section presents selected code snippets that highlight the important implementation details of the system. Only key parts of the code are included for demonstration purposes.

A.1 Flask Application Initialization

This code initializes the Flask application and connects it to the MySQL database using SQLAlchemy.

```
from flask import Flask

from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql+pymysql://root:@localhost/myeducationportal'

app.config['SECRET_KEY'] = 'secretkey'

db = SQLAlchemy(app)
```

Explanation:

- Initializes the Flask application.
- Connects the application to the MySQL database.
- SQLAlchemy is used as the ORM to manage database operations.

A.2 User Login Authentication

The following code handles the login process and verifies user credentials.

```
@app.route('/login', methods=['GET', 'POST'])

def login():

    username = request.form.get('username')

    password = request.form.get('password')

    user = Admin.query.filter_by(username=username).first()

    if user and check_password_hash(user.password, password):

        session['user'] = username

        return redirect('/dashboard')

    else:

        flash("Invalid Username or Password")

        return redirect('/login')
```

Explanation:

- Retrieves login credentials entered by the user.

- Validates credentials using database records.

A.3 Student Registration Model

This code defines the structure of the **Student table** in the database.

```
class Student(db.Model):  
    student_id = db.Column(db.Integer, primary_key=True)  
    roll_no = db.Column(db.Integer, unique=True)  
    student_name = db.Column(db.String(80))  
    gender = db.Column(db.String(10))  
    dob = db.Column(db.Date)  
    standard_id = db.Column(db.Integer, db.ForeignKey('standard.standard_id'))
```

Explanation:

- Defines the schema of the Student table.
- Each student has a unique student ID.
- The table stores student personal information and class reference.

A.4 Attendance Recording Logic

The following code records student attendance in the database.

```
@app.route('/mark_attendance', methods=['POST'])  
def mark_attendance():  
    student_id = request.form.get('student_id')  
    date = request.form.get('date')  
    status = request.form.get('status')  
    attendance = Attendance(  
        student_id=student_id,  
        date=date,  
        status=status  
    )  
    db.session.add(attendance)  
    db.session.commit()  
    return redirect('/attendance')
```

Explanation:

- Receives attendance details from the admin interface.
- Creates a new attendance record.
- Stores the record in the database.

A.5 Performance Entry Code

This code allows the administrator to record student marks.

```
@app.route('/add_marks', methods=['POST'])
def add_marks():
    student_id = request.form.get('student_id')
    subject = request.form.get('subject')
    marks = request.form.get('marks')
    performance = Performance(
        student_id=student_id,
        subject=subject,
        marks=marks
    )
    db.session.add(performance)
    db.session.commit()
    return redirect('/performance')
```

Explanation:

- Allows administrators to add academic performance data.
- The marks are stored in the Performance table.

A.6 Student Search API

This API allows searching students dynamically.

```
@app.route('/api/search_students')
def search_students():

    query = request.args.get('q')

    students = Student.query.filter(
        Student.student_name.ilike(f"%{query}%")
    ).all()
    return jsonify([student.student_name for student in students])
```

Explanation:

- Accepts search queries from the user interface.
- Retrieves matching student records from the database.

APPENDIX – B

Sample Database Tables

Student Table

Student ID	Name	Roll No	Class
101	Rahul Sharma	01	TYIT
102	Priya Patel	02	TYIT

Attendance Table

Attendance ID	Student ID	Date	Status
01	101	12-02-2026	Present
02	102	12-02-2026	Absent

Performance Table

Performance ID	Student ID	Subject	Marks
01	101	DBMS	85
02	102	DBMS	78